

LABORATORY MANUAL

For

DATABASE MANAGEMENT SYSTEMS

Course Code: U25DSPC307

S.Y. BTECH

SEMESTER-III

ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

PRACTICAL NO. 6

NESTED QUERIES AND CTEs

1. Aim

To write nested queries (subqueries), Common Table Expressions (CTEs) and recursive CTEs in MySQL.

2. Problem Statement

Using the *college_db* sample database, answer the following:

1. Instructors earning more than the average salary; CS101 students scoring above the CS101 average.
2. Students taught by instructor 101 (nested IN) and courses with no enrollment (NOT IN).
3. Instructors earning more than every AIML instructor (ALL) and non-AIML instructors earning less than some AIML instructor (ANY).
4. Enrollments above their own course average (correlated), departments without students (NOT EXISTS) and students who scored 90 or more in some course (EXISTS).
5. Students whose average is at least 80 (derived table) and the number of courses and students of each department (subqueries in SELECT).
6. Courses whose average exceeds 78 and the department with the best average (CTEs).
7. Generate semesters 1-8, list the full prerequisite chain of Deep Learning and display the mentoring hierarchy of instructors (recursive CTEs).

Note: The course document lists only the title of this practical. The case study, sample data and individual tasks below were prepared for this report to fulfil that title; they are illustrative and are not official university questions. All names, e-mail addresses and phone numbers are fictitious.

3. Objectives

1. To write single-row, multiple-row and nested subqueries.
2. To use IN, NOT IN, ANY, ALL, EXISTS and NOT EXISTS with subqueries.
3. To understand and write correlated subqueries and derived tables.
4. To simplify complex queries using non-recursive CTEs.
5. To process hierarchical data using recursive CTEs.

4. Software / Tools Required

Sr. No.	Software / Tool	Purpose
1	MySQL Community Server 8.0 (version 8.0.46 used)	Relational DBMS on which all SQL statements were executed
2	MySQL command-line client (mysql)	Running the SQL scripts and recording the output
3	Operating system: Ubuntu 24.04 LTS (Windows 10/11 also suitable)	Platform for the DBMS server and client
4	Text editor (VS Code / Notepad++)	Writing and saving the .sql script files

5. Theory

5.1 Subqueries (Nested Queries)

A **subquery** is a SELECT statement written inside another SQL statement, enclosed in parentheses. The inner query supplies values that the outer query uses. Subqueries can appear in WHERE, HAVING, FROM (derived table) and the SELECT list, and also inside INSERT, UPDATE and DELETE.

Type	Description	Operators / example
Scalar (single-row)	Returns exactly one value	=, <, > ... (SELECT AVG(salary) ...)
Multiple-row	Returns one column with many rows	IN, NOT IN, ANY / SOME, ALL
Correlated	Inner query refers to a column of the outer query; re-evaluated per outer row	WHERE e.marks > (SELECT AVG(...) WHERE e2.course_id = e.course_id)
EXISTS / NOT EXISTS	True if the subquery returns at least one / no row	WHERE NOT EXISTS (SELECT 1 ...)
Derived table	Subquery in FROM; must have an alias	FROM (SELECT ...) AS t

Caution with NOT IN: if the subquery returns a NULL, $\times$ **NOT IN** (. . .) is never TRUE and the query returns no rows; NOT EXISTS does not have this problem. $>$ **ALL** (. . .) means greater than the maximum and $<$ **ANY** (. . .) means less than the maximum value returned.

5.2 Common Table Expressions (CTEs)

A CTE is a named temporary result set defined with the WITH clause that exists only for the duration of one statement. CTEs make long queries readable by splitting them into named steps, and a CTE may be referenced several times in the main query. MySQL supports CTEs from version 8.0.

```
WITH cte1 AS (SELECT ...),
     cte2 AS (SELECT ... FROM cte1 ...)
SELECT ... FROM cte1 JOIN cte2 ...;
```

5.3 Recursive CTEs

A recursive CTE refers to itself and is used for hierarchical or graph data such as organisation charts, bills of materials and course prerequisites. It has two parts joined by UNION ALL (or UNION):

- **Anchor member** - a non-recursive SELECT that produces the initial rows.

- **Recursive member** - a SELECT that references the CTE name and produces new rows from the rows of the previous iteration. Recursion stops when an iteration produces no new rows.

```
WITH RECURSIVE cte_name (columns) AS (
    anchor_select
    UNION ALL
    recursive_select_referencing_cte_name
)
SELECT ... FROM cte_name;
```

MySQL limits the number of iterations with the system variable `cte_max_recursion_depth` (default 1000) to stop runaway recursion. **DBMS differences:** PostgreSQL and MySQL require the keyword RECURSIVE; SQL Server and Oracle (11g R2 and later) write only WITH; Oracle also offers the older CONNECT BY PRIOR syntax for hierarchies; SQL Server's limit is OPTION (MAXRECURSION n).

6. Procedure / Algorithm

1. Create and populate *college_db* (Q1, Q2).
2. Write scalar subqueries that compare a column with an aggregate computed by the inner query.
3. Write multiple-row subqueries using IN, NOT IN, ANY and ALL, including a nested subquery of three levels.
4. Write correlated subqueries and use EXISTS / NOT EXISTS to test for related rows.
5. Use a subquery in the FROM clause (derived table) and in the SELECT list.
6. Rewrite multi-step logic using one and several CTEs in a WITH clause.
7. Write recursive CTEs: a number generator, the prerequisite chain of a course and the mentoring hierarchy of instructors.
8. Observe how MySQL stops a non-terminating recursive CTE using `cte_max_recursion_depth`.
9. Verify every result against the sample data and record the output.

7. SQL Queries / Program

Q1: Setup: create college_db and its tables (from Practical 2)

```
DROP DATABASE IF EXISTS college_db;
CREATE DATABASE college_db;
USE college_db;

CREATE TABLE department (
    dept_id      INT          NOT NULL,
    dept_code    VARCHAR(10)  NOT NULL,
    dept_name    VARCHAR(40)  NOT NULL,
    building     VARCHAR(20),
    budget       DECIMAL(12,2) DEFAULT 1000000.00,
    CONSTRAINT pk_department PRIMARY KEY (dept_id),
    CONSTRAINT uq_dept_code  UNIQUE (dept_code),
    CONSTRAINT chk_budget    CHECK (budget > 0)
);

CREATE TABLE instructor (
```

```

instr_id      INT          NOT NULL,
instr_name    VARCHAR(40)  NOT NULL,
designation   VARCHAR(20)  NOT NULL,
dept_id       INT,
salary        DECIMAL(10,2) NOT NULL,
email         VARCHAR(40)  NOT NULL,
mentor_id     INT,
CONSTRAINT pk_instructor PRIMARY KEY (instr_id),
CONSTRAINT uq_instr_email UNIQUE (email),
CONSTRAINT chk_salary    CHECK (salary BETWEEN 30000 AND 300000),
CONSTRAINT fk_instr_dept FOREIGN KEY (dept_id)
    REFERENCES department (dept_id) ON DELETE SET NULL ON UPDATE CASCADE,
CONSTRAINT fk_instr_mentor FOREIGN KEY (mentor_id)
    REFERENCES instructor (instr_id) ON DELETE SET NULL
);

CREATE TABLE student (
    student_id INT          NOT NULL AUTO_INCREMENT,
    roll_no     VARCHAR(10)  NOT NULL,
    student_name VARCHAR(40) NOT NULL,
    gender      CHAR(1)      NOT NULL,
    dob         DATE         NOT NULL,
    dept_id     INT          NOT NULL,
    city        VARCHAR(30)  DEFAULT 'Pune',
    email       VARCHAR(40)  NOT NULL,
    CONSTRAINT pk_student PRIMARY KEY (student_id),
    CONSTRAINT uq_roll_no  UNIQUE (roll_no),
    CONSTRAINT uq_student_email UNIQUE (email),
    CONSTRAINT chk_gender  CHECK (gender IN ('M', 'F', 'O')),
    CONSTRAINT chk_dob     CHECK (dob >= '1995-01-01'),
    CONSTRAINT fk_student_dept FOREIGN KEY (dept_id)
    REFERENCES department (dept_id) ON UPDATE CASCADE
);

CREATE TABLE student_phone (
    student_id INT          NOT NULL,
    phone      CHAR(10)     NOT NULL,
    CONSTRAINT pk_student_phone PRIMARY KEY (student_id, phone),
    CONSTRAINT chk_phone        CHECK (phone REGEXP '^[6-9][0-9]{9}$'),
    CONSTRAINT fk_phone_student FOREIGN KEY (student_id)
    REFERENCES student (student_id) ON DELETE CASCADE
);

CREATE TABLE course (
    course_id CHAR(5)      NOT NULL,
    title     VARCHAR(40)  NOT NULL,
    dept_id   INT          NOT NULL,
    credits   TINYINT      NOT NULL,
    instr_id  INT,
    prereq_id CHAR(5),
    CONSTRAINT pk_course PRIMARY KEY (course_id),
    CONSTRAINT uq_course_title UNIQUE (title),
    CONSTRAINT chk_credits CHECK (credits BETWEEN 1 AND 5),
    CONSTRAINT fk_course_dept FOREIGN KEY (dept_id)
    REFERENCES department (dept_id) ON UPDATE CASCADE,
    CONSTRAINT fk_course_instr FOREIGN KEY (instr_id)
    REFERENCES instructor (instr_id) ON DELETE SET NULL,
    CONSTRAINT fk_course_prereq FOREIGN KEY (prereq_id)
    REFERENCES course (course_id)
);

CREATE TABLE enrollment (
    student_id INT          NOT NULL,
    course_id  CHAR(5)      NOT NULL,
    semester   TINYINT      NOT NULL,
    marks      INT,
    grade      CHAR(2),
    CONSTRAINT pk_enrollment PRIMARY KEY (student_id, course_id),
    CONSTRAINT chk_semester CHECK (semester BETWEEN 1 AND 8),

```

```

CONSTRAINT chk_marks      CHECK (marks BETWEEN 0 AND 100),
CONSTRAINT fk_enr_student FOREIGN KEY (student_id)
    REFERENCES student (student_id) ON DELETE CASCADE,
CONSTRAINT fk_enr_course  FOREIGN KEY (course_id) REFERENCES course (course_id)
);

```

This setup script recreates the common *college_db* case-study database (schema of Practical 2 and data of Practical 3) so that this practical can be executed independently on a fresh MySQL server.

Q2: Setup: insert the sample data (from Practical 3)

```

INSERT INTO department (dept_id, dept_code, dept_name, building, budget) VALUES
(10, 'CSE', 'Computer Engineering', 'A Block', 2500000),
(20, 'AIML', 'AI and Machine Learning', 'B Block', 3000000),
(30, 'ECE', 'Electronics and Telecomm', 'C Block', 1800000),
(40, 'MECH', 'Mechanical Engineering', 'D Block', 1500000),
(50, 'CIVIL', 'Civil Engineering', 'D Block', 1200000);

INSERT INTO instructor VALUES
(101, 'Anil Kulkarni', 'Professor', 10, 185000, 'anil.k@example.com', NULL),
(102, 'Meera Joshi', 'Professor', 20, 172000, 'meera.j@example.com', 101),
(103, 'Rahul Patil', 'Asst. Prof', 10, 82000, 'rahul.p@example.com', 101),
(104, 'Sneha Deshmukh', 'Assoc. Prof', 20, 115000, 'sneha.d@example.com', 102),
(105, 'Vikram Shinde', 'Professor', 30, 168000, 'vikram.s@example.com', 101),
(106, 'Pooja Kale', 'Asst. Prof', 20, 78000, 'pooja.k@example.com', 104),
(107, 'Nitin Pawar', 'Asst. Prof', 50, 76000, 'nitin.p@example.com', 105),
(108, 'Kavita More', 'Assoc. Prof', 40, 112000, 'kavita.m@example.com', 105);

INSERT INTO student
(roll_no, student_name, gender, dob, dept_id, city, email) VALUES
('S24CS001', 'Aarav Sharma', 'M', '2005-03-14', 10, 'Pune', 'aarav@example.com'),
('S24CS002', 'Isha Gupta', 'F', '2005-07-22', 10, 'Mumbai', 'isha@example.com'),
('S24AI001', 'Rohan Mehta', 'M', '2005-01-09', 20, 'Pune', 'rohan@example.com'),
('S24AI002', 'Priya Nair', 'F', '2004-11-30', 20, 'Nashik', 'priya@example.com'),
('S24AI003', 'Kunal Verma', 'M', '2005-05-18', 20, 'Pune', 'kunal@example.com'),
('S24EC001', 'Sneha Iyer', 'F', '2005-09-02', 30, 'Nagpur', 'sneha@example.com'),
('S24EC002', 'Aditya Rao', 'M', '2004-12-25', 30, 'Mumbai', 'aditya@example.com'),
('S24ME001', 'Neha Kulkarni', 'F', '2005-02-11', 40, 'Pune', 'neha@example.com'),
('S24CS003', 'Yash Jadhav', 'M', '2005-08-05', 10, NULL, 'yash@example.com'),
('S24AI004', 'Ananya Desai', 'F', '2005-04-27', 20, 'Mumbai', 'ananya@example.com');

INSERT INTO student_phone VALUES
(1, '9876500011'), (1, '9876500012'), (2, '9876500021'), (3, '9876500031'),
(4, '9876500041'), (4, '9876500042'), (6, '9876500061'), (9, '9876500091');

INSERT INTO course VALUES
('CS101', 'Programming in C', 10, 4, 103, NULL),
('CS201', 'Data Structures', 10, 4, 103, 'CS101'),
('CS301', 'Database Management Systems', 10, 3, 101, 'CS201'),
('CS401', 'Advanced Databases', 10, 3, 101, 'CS301'),
('AI201', 'Python for Data Science', 20, 3, 104, 'CS101'),
('AI301', 'Machine Learning', 20, 4, 102, 'AI201'),
('AI401', 'Deep Learning', 20, 3, 106, 'AI301'),
('EC201', 'Digital Electronics', 30, 3, 105, NULL),
('EC301', 'Signals and Systems', 30, 4, NULL, 'EC201'),
('ME201', 'Engineering Mechanics', 40, 3, 108, NULL);

INSERT INTO enrollment (student_id, course_id, semester, marks, grade) VALUES
(1, 'CS101', 3, 78, 'A'), (1, 'CS201', 3, 85, 'A+'), (1, 'CS301', 4, 91, 'O'),
(1, 'AI201', 3, 72, 'A'), (2, 'CS101', 3, 88, 'A+'), (2, 'CS201', 3, 67, 'B+'),
(2, 'CS301', 4, 74, 'A'), (3, 'CS101', 3, 65, 'B+'), (3, 'AI201', 3, 81, 'A+'),
(3, 'AI301', 4, 89, 'A+'), (3, 'CS301', 4, 70, 'A'), (4, 'AI201', 3, 92, 'O'),
(4, 'AI301', 4, 76, 'A'), (4, 'AI401', 4, 84, 'A+'), (5, 'AI201', 3, 58, 'B'),
(5, 'AI301', 4, 63, 'B+'), (6, 'EC201', 3, 79, 'A'), (6, 'CS101', 3, 55, 'B'),
(6, 'EC301', 4, 82, 'A+'), (7, 'EC201', 3, 45, 'C'), (7, 'EC301', 4, 61, 'B+'),

```

```
(8, 'CS101', 3, 78, 'A'), (9, 'CS201', 3, 90, 'O'), (9, 'CS301', 4, 95, 'O'),
(9, 'CS401', 4, 88, 'A+');
```

Q3: Single-row (scalar) subqueries

```
SELECT instr_name, salary
FROM   instructor
WHERE  salary > (SELECT AVG(salary) FROM instructor)
ORDER BY salary DESC;

SELECT s.student_name, e.marks
FROM   enrollment e JOIN student s ON s.student_id = e.student_id
WHERE  e.course_id = 'CS101'
      AND e.marks > (SELECT AVG(marks) FROM enrollment WHERE course_id = 'CS101');
```

Q4: Multiple-row subqueries with IN and NOT IN

```
SELECT roll_no, student_name
FROM   student
WHERE  student_id IN (SELECT student_id FROM enrollment
                     WHERE course_id IN (SELECT course_id FROM course
                                         WHERE instr_id = 101));

SELECT course_id, title
FROM   course
WHERE  course_id NOT IN (SELECT course_id FROM enrollment);
```

Q5: Multiple-row subqueries with ANY and ALL

```
SELECT instr_name, dept_id, salary
FROM   instructor
WHERE  salary > ALL (SELECT salary FROM instructor WHERE dept_id = 20);

SELECT instr_name, dept_id, salary
FROM   instructor
WHERE  dept_id <> 20
      AND salary < ANY (SELECT salary FROM instructor WHERE dept_id = 20);
```

Q6: Correlated subqueries, EXISTS and NOT EXISTS

```
SELECT e.student_id, e.course_id, e.marks
FROM   enrollment e
WHERE  e.marks > (SELECT AVG(e2.marks) FROM enrollment e2
                 WHERE e2.course_id = e.course_id)
ORDER BY e.course_id, e.marks DESC;

SELECT d.dept_code, d.dept_name
FROM   department d
WHERE  NOT EXISTS (SELECT 1 FROM student s WHERE s.dept_id = d.dept_id);

SELECT s.roll_no, s.student_name
FROM   student s
WHERE  EXISTS (SELECT 1 FROM enrollment e
               WHERE e.student_id = s.student_id AND e.marks >= 90);
```

In a correlated subquery the inner query refers to a column of the outer query (e.course_id), so it is logically evaluated once for every row of the outer query.

Q7: Subquery in FROM (derived table) and in the SELECT list

```

SELECT t.student_id, t.avg_marks
FROM   (SELECT student_id, ROUND(AVG(marks), 2) AS avg_marks
        FROM   enrollment GROUP BY student_id) AS t
WHERE  t.avg_marks >= 80;

SELECT d.dept_code,
       (SELECT COUNT(*) FROM course c WHERE c.dept_id = d.dept_id) AS courses,
       (SELECT COUNT(*) FROM student s WHERE s.dept_id = d.dept_id) AS students
FROM   department d
ORDER BY d.dept_id;

```

Q8: Common Table Expressions (WITH): single and multiple CTEs

```

WITH course_stats AS (
    SELECT course_id, ROUND(AVG(marks), 2) AS avg_marks, MAX(marks) AS top_marks
    FROM   enrollment
    GROUP BY course_id
)
SELECT c.course_id, c.title, cs.avg_marks, cs.top_marks
FROM   course c JOIN course_stats cs ON cs.course_id = c.course_id
WHERE  cs.avg_marks > 78;

WITH dept_avg AS (
    SELECT c.dept_id, AVG(e.marks) AS avg_marks
    FROM   enrollment e JOIN course c ON c.course_id = e.course_id
    GROUP BY c.dept_id
),
best AS (
    SELECT MAX(avg_marks) AS best_avg FROM dept_avg
)
SELECT d.dept_name, ROUND(da.avg_marks, 2) AS avg_marks
FROM   dept_avg da
JOIN   best b      ON da.avg_marks = b.best_avg
JOIN   department d ON d.dept_id = da.dept_id;

```

Q9: Recursive CTE: generate the numbers 1 to 8 (semester list)

```

WITH RECURSIVE sem (n) AS (
    SELECT 1 -- anchor member
    UNION ALL
    SELECT n + 1 FROM sem WHERE n < 8 -- recursive member
)
SELECT n AS semester,
       IF(n % 2 = 1, 'Odd (Jul-Dec)', 'Even (Jan-Jun)') AS term
FROM   sem;

```

Q10: Recursive CTE: complete prerequisite chain of Deep Learning (AI401)

```

WITH RECURSIVE prereq_chain AS (
    SELECT course_id, title, prereq_id, 0 AS level
    FROM   course
    WHERE  course_id = 'AI401'
    UNION ALL
    SELECT c.course_id, c.title, c.prereq_id, pc.level + 1
    FROM   course c
    JOIN   prereq_chain pc ON c.course_id = pc.prereq_id
)
SELECT level, course_id, title, prereq_id
FROM   prereq_chain
ORDER BY level;

```


Q11: Recursive CTE: mentoring hierarchy of instructors with level and path

```

WITH RECURSIVE hierarchy AS (
    SELECT instr_id, instr_name, mentor_id, 1 AS level,
           CAST(instr_name AS CHAR(200)) AS path
    FROM   instructor
    WHERE  mentor_id IS NULL
    UNION ALL
    SELECT i.instr_id, i.instr_name, i.mentor_id, h.level + 1,
           CONCAT(h.path, ' > ', i.instr_name)
    FROM   instructor i
    JOIN   hierarchy h ON i.mentor_id = h.instr_id
)
SELECT level, CONCAT(REPEAT('   ', level - 1), instr_name) AS instructor, path
FROM   hierarchy
ORDER BY path;

```

The anchor member is cast to CHAR(200) because the column type of a recursive CTE is fixed by the anchor member; without the cast the growing path would be truncated.

Q12: Recursion limit: a recursive CTE without a terminating condition

```

SELECT @@cte_max_recursion_depth;
WITH RECURSIVE endless (n) AS (
    SELECT 1 UNION ALL SELECT n + 1 FROM endless
)
SELECT COUNT(*) FROM endless;

```

8. Sample Input / Database Tables

The following tables of *college_db* (created by Q1 and Q2) are used. The values were read from the database after executing the setup queries. The prereq_id column of course and the mentor_id column of instructor form the hierarchies processed by the recursive CTEs.

Table: department

dept_id	dept_code	dept_name	building	budget
10	CSE	Computer Engineering	A Block	2500000.00
20	AIML	AI and Machine Learning	B Block	3000000.00
30	ECE	Electronics and Telecomm	C Block	1800000.00
40	MECH	Mechanical Engineering	D Block	1500000.00
50	CIVIL	Civil Engineering	D Block	1200000.00

Table: instructor

instr_id	instr_name	designation	dept_id	salary	email	mentor_id
101	Anil Kulkarni	Professor	10	185000.00	anil.k@example.com	NULL
102	Meera Joshi	Professor	20	172000.00	meera.j@example.com	101
103	Rahul Patil	Asst. Prof	10	82000.00	rahul.p@example.com	101
104	Sneha Deshmukh	Assoc. Prof	20	115000.00	sneha.d@example.com	102
105	Vikram Shinde	Professor	30	168000.00	vikram.s@example.com	101
106	Pooja Kale	Asst. Prof	20	78000.00	pooja.k@example.com	104
107	Nitin Pawar	Asst. Prof	50	76000.00	nitin.p@example.com	105
108	Kavita More	Assoc. Prof	40	112000.00	kavita.m@example.com	105

Table: student

student_id	roll_no	student_name	gender	dob	dept_id	city	email
1	S24CS001	Aarav Sharma	M	2005-03-14	10	Pune	aarav@example.com
2	S24CS002	Isha Gupta	F	2005-07-22	10	Mumbai	isha@example.com
3	S24AI001	Rohan Mehta	M	2005-01-09	20	Pune	rohan@example.com
4	S24AI002	Priya Nair	F	2004-11-30	20	Nashik	priya@example.com
5	S24AI003	Kunal Verma	M	2005-05-18	20	Pune	kunal@example.com
6	S24EC001	Sneha Iyer	F	2005-09-02	30	Nagpur	sneha@example.com
7	S24EC002	Aditya Rao	M	2004-12-25	30	Mumbai	aditya@example.com
8	S24ME001	Neha Kulkarni	F	2005-02-11	40	Pune	neha@example.com
9	S24CS003	Yash Jadhav	M	2005-08-05	10	NULL	yash@example.com
10	S24AI004	Ananya Desai	F	2005-04-27	20	Mumbai	ananya@example.com

Table: course

course_id	title	dept_id	credits	instr_id	prereq_id
AI201	Python for Data Science	20	3	104	CS101
AI301	Machine Learning	20	4	102	AI201
AI401	Deep Learning	20	3	106	AI301
CS101	Programming in C	10	4	103	NULL
CS201	Data Structures	10	4	103	CS101
CS301	Database Management Systems	10	3	101	CS201
CS401	Advanced Databases	10	3	101	CS301
EC201	Digital Electronics	30	3	105	NULL
EC301	Signals and Systems	30	4	NULL	EC201
ME201	Engineering Mechanics	40	3	108	NULL

Table: enrollment (25 rows)

student_id	course_id	semester	marks	grade
1	AI201	3	72	A
1	CS101	3	78	A
1	CS201	3	85	A+
1	CS301	4	91	O
2	CS101	3	88	A+
2	CS201	3	67	B+
2	CS301	4	74	A
3	AI201	3	81	A+
3	AI301	4	89	A+
3	CS101	3	65	B+

student_id	course_id	semester	marks	grade
3	CS301	4	70	A
4	AI201	3	92	O
4	AI301	4	76	A
4	AI401	4	84	A+
5	AI201	3	58	B
5	AI301	4	63	B+
6	CS101	3	55	B
6	EC201	3	79	A
6	EC301	4	82	A+
7	EC201	3	45	C
7	EC301	4	61	B+
8	CS101	3	78	A
9	CS201	3	90	O
9	CS301	4	95	O
9	CS401	4	88	A+

9. Output

The output below is the actual text produced by the MySQL 8.0.46 command-line client when the statements of Section 7 were executed in batch mode (`mysql --table -vvv < script.sql`). Each result is preceded by the statement that produced it; statements longer than a few lines are abbreviated with '...'. Because the script ran in batch mode, error messages report the line number within the script file. Execution times shown in brackets depend on the machine.

Output of Q1: Setup: create college_db and its tables (from Practical 2)

```
mysql> DROP DATABASE IF EXISTS college_db;
Query OK, 0 rows affected, 1 warning (0.00 sec)

mysql> CREATE DATABASE college_db;
Query OK, 1 row affected (0.00 sec)

mysql> CREATE TABLE department ( ...;
Query OK, 0 rows affected (0.01 sec)

mysql> CREATE TABLE instructor ( ...;
Query OK, 0 rows affected (0.02 sec)

mysql> CREATE TABLE student ( ...;
Query OK, 0 rows affected (0.01 sec)

mysql> CREATE TABLE student_phone ( ...;
Query OK, 0 rows affected (0.01 sec)

mysql> CREATE TABLE course ( ...;
Query OK, 0 rows affected (0.02 sec)

mysql> CREATE TABLE enrollment ( ...;
Query OK, 0 rows affected (0.01 sec)
```

Output of Q2: Setup: insert the sample data (from Practical 3)

```
mysql> INSERT INTO department (dept_id, dept_code, dept_name, building, budget) VALUES ...;
Query OK, 5 rows affected (0.00 sec)
Records: 5 Duplicates: 0 Warnings: 0

mysql> INSERT INTO instructor VALUES ...;
Query OK, 8 rows affected (0.01 sec)
Records: 8 Duplicates: 0 Warnings: 0

mysql> INSERT INTO student ...;
Query OK, 10 rows affected (0.00 sec)
Records: 10 Duplicates: 0 Warnings: 0

mysql> INSERT INTO student_phone VALUES ...;
Query OK, 8 rows affected (0.00 sec)
Records: 8 Duplicates: 0 Warnings: 0

mysql> INSERT INTO course VALUES ...;
Query OK, 10 rows affected (0.00 sec)
Records: 10 Duplicates: 0 Warnings: 0

mysql> INSERT INTO enrollment (student_id, course_id, semester, marks, grade) VALUES ...;
Query OK, 25 rows affected (0.00 sec)
Records: 25 Duplicates: 0 Warnings: 0
```

Output of Q3: Single-row (scalar) subqueries

```
mysql> SELECT instr_name, salary
-> FROM instructor
-> WHERE salary > (SELECT AVG(salary) FROM instructor)
-> ORDER BY salary DESC;
+-----+-----+
| instr_name | salary |
+-----+-----+
| Anil Kulkarni | 185000.00 |
| Meera Joshi | 172000.00 |
| Vikram Shinde | 168000.00 |
+-----+-----+
3 rows in set (0.01 sec)

mysql> SELECT s.student_name, e.marks
-> FROM enrollment e JOIN student s ON s.student_id = e.student_id
-> WHERE e.course_id = 'CS101'
-> AND e.marks > (SELECT AVG(marks) FROM enrollment WHERE course_id = 'CS101');
+-----+-----+
| student_name | marks |
+-----+-----+
| Aarav Sharma | 78 |
| Isha Gupta | 88 |
| Neha Kulkarni | 78 |
+-----+-----+
3 rows in set (0.00 sec)
```

Output of Q4: Multiple-row subqueries with IN and NOT IN

```
mysql> SELECT roll_no, student_name
-> FROM student
-> WHERE student_id IN (SELECT student_id FROM enrollment
-> WHERE course_id IN (SELECT course_id FROM course
-> WHERE instr_id = 101));
```

roll_no	student_name
S24CS001	Aarav Sharma
S24CS002	Isha Gupta
S24AI001	Rohan Mehta
S24CS003	Yash Jadhav

4 rows in set (0.00 sec)

```
mysql> SELECT course_id, title
-> FROM course
-> WHERE course_id NOT IN (SELECT course_id FROM enrollment);
```

course_id	title
ME201	Engineering Mechanics

1 row in set (0.00 sec)

Output of Q5: Multiple-row subqueries with ANY and ALL

```
mysql> SELECT instr_name, dept_id, salary
-> FROM instructor
-> WHERE salary > ALL (SELECT salary FROM instructor WHERE dept_id = 20);
```

instr_name	dept_id	salary
Anil Kulkarni	10	185000.00

1 row in set (0.00 sec)

```
mysql> SELECT instr_name, dept_id, salary
-> FROM instructor
-> WHERE dept_id <> 20
-> AND salary < ANY (SELECT salary FROM instructor WHERE dept_id = 20);
```

instr_name	dept_id	salary
Rahul Patil	10	82000.00
Vikram Shinde	30	168000.00
Kavita More	40	112000.00
Nitin Pawar	50	76000.00

4 rows in set (0.00 sec)

Output of Q6: Correlated subqueries, EXISTS and NOT EXISTS

```
mysql> SELECT e.student_id, e.course_id, e.marks
-> FROM enrollment e
-> WHERE e.marks > (SELECT AVG(e2.marks) FROM enrollment e2
-> WHERE e2.course_id = e.course_id)
-> ORDER BY e.course_id, e.marks DESC;
```

student_id	course_id	marks
4	AI201	92
3	AI201	81
3	AI301	89
2	CS101	88
1	CS101	78
8	CS101	78
9	CS201	90

1	CS201	85
9	CS301	95
1	CS301	91
6	EC201	79
6	EC301	82

```
+-----+-----+
12 rows in set (0.00 sec)
```

```
mysql> SELECT d.dept_code, d.dept_name
-> FROM department d
-> WHERE NOT EXISTS (SELECT 1 FROM student s WHERE s.dept_id = d.dept_id);
```

dept_code	dept_name
CIVIL	Civil Engineering

```
+-----+-----+
1 row in set (0.00 sec)
```

```
mysql> SELECT s.roll_no, s.student_name
-> FROM student s
-> WHERE EXISTS (SELECT 1 FROM enrollment e
-> WHERE e.student_id = s.student_id AND e.marks >= 90);
```

roll_no	student_name
S24CS001	Aarav Sharma
S24AI002	Priya Nair
S24CS003	Yash Jadhav

```
+-----+-----+
3 rows in set (0.00 sec)
```

Output of Q7: Subquery in FROM (derived table) and in the SELECT list

```
mysql> SELECT t.student_id, t.avg_marks
-> FROM (SELECT student_id, ROUND(AVG(marks), 2) AS avg_marks
-> FROM enrollment GROUP BY student_id) AS t
-> WHERE t.avg_marks >= 80;
```

student_id	avg_marks
1	81.50
4	84.00
9	91.00

```
+-----+-----+
3 rows in set (0.00 sec)
```

```
mysql> SELECT d.dept_code,
-> (SELECT COUNT(*) FROM course c WHERE c.dept_id = d.dept_id) AS courses,
-> (SELECT COUNT(*) FROM student s WHERE s.dept_id = d.dept_id) AS students
-> FROM department d
-> ORDER BY d.dept_id;
```

dept_code	courses	students
CSE	4	3
AIML	3	4
ECE	2	2
MECH	1	1
CIVIL	0	0

```
+-----+-----+
5 rows in set (0.00 sec)
```

Output of Q8: Common Table Expressions (WITH): single and multiple CTEs

```
mysql> WITH course_stats AS (
->   SELECT course_id, ROUND(AVG(marks), 2) AS avg_marks, MAX(marks) AS top_marks
->   FROM   enrollment
->   GROUP BY course_id
-> )
-> SELECT c.course_id, c.title, cs.avg_marks, cs.top_marks
-> FROM   course c JOIN course_stats cs ON cs.course_id = c.course_id
-> WHERE  cs.avg_marks > 78;
```

course_id	title	avg_marks	top_marks
CS401	Advanced Databases	88.00	88
CS201	Data Structures	80.67	90
CS301	Database Management Systems	82.50	95
AI401	Deep Learning	84.00	84

4 rows in set (0.00 sec)

```
mysql> WITH dept_avg AS (
->   SELECT c.dept_id, AVG(e.marks) AS avg_marks
->   FROM   enrollment e JOIN course c ON c.course_id = e.course_id
->   GROUP BY c.dept_id
-> ...
-> JOIN   department d ON d.dept_id = da.dept_id;
```

dept_name	avg_marks
Computer Engineering	78.77

1 row in set (0.00 sec)

Output of Q9: Recursive CTE: generate the numbers 1 to 8 (semester list)

```
mysql> WITH RECURSIVE sem (n) AS (
->   SELECT 1
->   UNION ALL
->   SELECT n + 1 FROM sem WHERE n < 8
-> )
-> SELECT n AS semester,
->   IF(n % 2 = 1, 'Odd (Jul-Dec)', 'Even (Jan-Jun)') AS term
-> FROM   sem;
```

semester	term
1	Odd (Jul-Dec)
2	Even (Jan-Jun)
3	Odd (Jul-Dec)
4	Even (Jan-Jun)
5	Odd (Jul-Dec)
6	Even (Jan-Jun)
7	Odd (Jul-Dec)
8	Even (Jan-Jun)

8 rows in set (0.00 sec)

Output of Q10: Recursive CTE: complete prerequisite chain of Deep Learning (AI401)

```
mysql> WITH RECURSIVE prereq_chain AS (
->   SELECT course_id, title, prereq_id, 0 AS level
->   FROM   course
->   WHERE  course_id = 'AI401'
->   ...
-> ORDER BY level;
+-----+-----+-----+-----+
| level | course_id | title                | prereq_id |
+-----+-----+-----+-----+
| 0     | AI401     | Deep Learning        | AI301     |
| 1     | AI301     | Machine Learning     | AI201     |
| 2     | AI201     | Python for Data Science | CS101     |
| 3     | CS101     | Programming in C      | NULL      |
+-----+-----+-----+-----+
4 rows in set (0.00 sec)
```

Output of Q11: Recursive CTE: mentoring hierarchy of instructors with level and path

```
mysql> WITH RECURSIVE hierarchy AS (
->   SELECT instr_id, instr_name, mentor_id, 1 AS level,
->         CAST(instr_name AS CHAR(200)) AS path
->   FROM   instructor
->   ...
-> ORDER BY path;
+-----+-----+-----+-----+
| level | instructor          | path                                     |
+-----+-----+-----+-----+
| 1     | Anil Kulkarni        | Anil Kulkarni                         |
| 2     | Meera Joshi          | Anil Kulkarni > Meera Joshi           |
| 3     | Sneha Deshmukh       | Anil Kulkarni > Meera Joshi > Sneha Deshmukh |
| 4     | Pooja Kale           | Anil Kulkarni > Meera Joshi > Sneha Deshmukh > Pooja Kale |
| 2     | Rahul Patil          | Anil Kulkarni > Rahul Patil           |
| 2     | Vikram Shinde        | Anil Kulkarni > Vikram Shinde         |
| 3     | Kavita More          | Anil Kulkarni > Vikram Shinde > Kavita More |
| 3     | Nitin Pawar          | Anil Kulkarni > Vikram Shinde > Nitin Pawar |
+-----+-----+-----+-----+
8 rows in set (0.00 sec)
```

Output of Q12: Recursion limit: a recursive CTE without a terminating condition

```
mysql> SELECT @@cte_max_recursion_depth;
+-----+
| @@cte_max_recursion_depth |
+-----+
| 1000 |
+-----+
1 row in set (0.00 sec)

mysql> WITH RECURSIVE endless (n) AS (
->   SELECT 1 UNION ALL SELECT n + 1 FROM endless
-> )
-> SELECT COUNT(*) FROM endless;
ERROR 3636 (HY000) at line 264: Recursive query aborted after 1001 iterations. Try increasing @@cte_max_recursion_depth to a larger value.
```


10. Result

All subqueries and CTEs executed successfully on MySQL 8.0. Scalar, multiple-row (IN, NOT IN, ANY, ALL), correlated and EXISTS subqueries returned the expected rows (e.g. ME201 has no enrollment and CIVIL has no students). CTEs computed course statistics and the best department. Recursive CTEs generated semesters 1-8, the prerequisite chain AI401 - AI301 - AI201 - CS101 and the four-level mentoring hierarchy, and MySQL aborted the non-terminating recursive CTE after reaching `cte_max_recursion_depth` (1000).

11. Conclusion

Subqueries allow a query to use the result of another query, which makes it possible to compare rows with aggregates and to test for the existence of related rows. Correlated subqueries are powerful but are evaluated per outer row, so joins or CTEs are often clearer and faster. CTEs give names to intermediate results and make complex queries readable, and recursive CTEs provide a standard way to traverse hierarchies of unknown depth - something that cannot be done with a fixed number of self joins.